

University of Wollongong in Dubai
CSCI435 – Computer Vision Algorithms and Systems
Project Description

Dr. Patrick Mukala

Submission and Defence: Week 11

1 Context

Computer vision encompasses low-level, mid-level, and high-level vision tasks including image formation, edge detection, segmentation, object detection and recognition, face recognition, video processing, and scene understanding. This project requires students to construct a deployable application that integrates multiple computer vision techniques into a unified system. The application serves as an envelope around the core concepts presented by Dr. Patrick Mukala throughout the course. The objective is to demonstrate not only the operation of individual algorithms but also their collective functionality within a user-facing system.

2 Assignment

Each group must design and implement a holistic system, realised as either a web application or a mobile application, that integrates a minimum of four distinct computer vision capabilities drawn from the topics covered in CSCI435. The system must accept input from a camera, either as a live feed or as uploaded images or video files, and produce outputs that address a defined problem. Groups may select any application domain, including but not limited to security, healthcare, autonomous systems, retail, accessibility, or industrial inspection.

2.1 Permissible Vision Capabilities

The system must incorporate at least four capabilities from the following list:

- Image enhancement techniques
- Edge detection
- Keypoint detection
- Shape detection

- Object detection
- Object recognition
- Face detection or face recognition
- Video processing and moving object detection
- Change detection and background modelling
- Object tracking
- Image segmentation
- Binary morphological operations

2.2 Example Applications

- **Surveillance System:** motion detection (change detection), person tracking (object tracking), and face recognition for access control.
- **Assistive Navigation Tool:** edge detection for obstacle mapping, object detection for sign recognition, and video processing for real-time guidance.
- **Retail Analytics System:** object detection for product identification, face detection for customer counting, and keypoint detection for gesture recognition.

3 Requirements

3.1 Functional Requirements

1. The system must accept real-time camera input or uploaded images and video files.
2. A minimum of four distinct computer vision tasks must be integrated into a single workflow.
3. Results must be displayed to the user using bounding boxes, text overlays, or audio feedback.
4. The system must handle at least two input modalities, for example a single image and a live video stream.
5. A user story must be documented, explaining how the selected vision capabilities function together.

3.2 Technical Requirements

1. For a web application: HTML, CSS, and JavaScript with a framework such as React, Vue, Streamlit, or Gradio. For a mobile application: Flutter, React Native, or native Android or iOS development.
2. A backend implemented in Python with FastAPI, Flask, or Django is permitted. On-device inference using TensorFlow Lite, MediaPipe, or Core ML is permitted.
3. At least one pre-trained model may be used. At least one model must be fine-tuned or trained on custom data.
4. The application must be demonstrable live during the defence. Local hosting is acceptable.
5. All code must be shared via GitHub or GitLab with a README file containing clear execution instructions.

3.3 Group Work Requirements

- Groups shall consist of four to five students. Students are not required to be enrolled in the same laboratory session.
- Each group member must contribute to coding, documentation, and presentation. A contribution table specifying each member's tasks is required in the report.

4 Submission and Defence

4.1 Report

The report must be submitted as a PDF document via Moodle and contain the following sections:

- Title page: project name, group members, student identification numbers, subject code (CSCI435), subject name, and date.
- Introduction: problem statement, user story, and selected vision capabilities.
- System architecture: diagram and description of frontend, backend, models, and data flow.
- Implementation details: key code snippets, model selection, training or fine-tuning steps, and challenges encountered.
- Experiments and results: quantitative metrics including accuracy, frames per second, and latency, together with qualitative examples such as screenshots and sample outputs.

- Individual contributions: a table specifying each member's tasks.
- Conclusion and future work.
- References: papers, pre-trained models, and libraries used.

4.2 Code Submission

- A GitHub or GitLab repository link must be provided. The repository shall be made accessible to the lecturer.
- The repository must contain all source code with a clear folder structure, a requirements.txt or equivalent dependency file, and execution instructions in the README.
- A demonstration video of two to three minutes duration is required.

4.3 Live Defence

Defence will take place during laboratory sessions in Week 11. Each group will be allocated ten to twelve minutes: seven to eight minutes for a live demonstration followed by four minutes for questions and answers. The demonstration must be conducted live; pre-recorded videos are not accepted except as a backup in case of technical failure. All group members must speak and answer questions.

4.4 Deadline and Late Penalty

The report and GitHub or GitLab link must be submitted one day before the scheduled Week 11 laboratory session via Moodle. A penalty of twenty per cent per working day applies to late submissions, including weekends, as specified in the CSCI435 subject outline.

5 Marking Criteria

The project contributes to the Class Activities (Project) component of CSCI435, weighted at 15% of the final subject grade. The project is assessed out of 100 marks according to the criteria in Table 2.

Criterion	Weight	Description
Integration of four or more vision tasks	15	Each task operates correctly and contributes to the user story.
<i>Continued</i>		

Criterion	Weight	Description
Correctness and robustness	15	The system handles edge cases, varying lighting conditions, and orientations.
User interface and user experience	10	The interface is intuitive and responsive, and vision results are clearly presented.
Model selection and training effort	10	At least one model is fine-tuned or trained on custom data. Model choices are justified.
System architecture and code quality	10	Code is modular, clean, and documented. Version control is properly used.
Performance	5	Real-time or near real-time performance is achieved, defined as a minimum of ten frames per second for video processing.
Report quality	15	The report is clear, complete, contains diagrams, presents quantitative results, and includes critical analysis.
Live demonstration and defence	15	The live demonstration succeeds, technical issues are handled appropriately, and questions are answered correctly.
Teamwork and contribution	5	Contributions are balanced, roles are clearly divided, and evidence of collaboration is present.

Table 2: Marking criteria (total 100 marks)

5.1 Pass and Fail Conditions

A passing grade for this project requires a minimum of 50 marks. A failing grade will be assigned under any of the following conditions:

- No working integrated system is submitted (only isolated scripts are provided).
- Fewer than three vision tasks are implemented.
- A live demonstration cannot be conducted during the defence.

6 Alignment with CSCI435 Learning Outcomes

This project assesses the following learning outcomes as specified in the subject outline:

- **LO2:** Use image enhancement techniques.
- **LO3:** Use object detection and recognition techniques.
- **LO4:** Use video processing techniques to detect moving objects.
- **LO5:** Design and implement basic computer vision systems for real applications.

7 Deliverables

The following items must be submitted by the specified deadline:

- . **Report:** A PDF document following the structure outlined in Section 4.1.
- . **Code Repository:** A GitHub or GitLab link containing all source code, dependency files, and a README with execution instructions.
- . **Demonstration Video:** A two to three minute video showing the system in operation.
- . **Live Defence Presentation:** A seven to eight minute live demonstration during the Week 11 laboratory session, followed by a four minute question and answer period.

8 Recommended Tools and Libraries

The following tools and libraries are recommended for implementation:

- Models and computer vision: OpenCV, PyTorch, TensorFlow, MediaPipe, YOLO (Ultralytics), scikit-image, Dlib.
- Web frontend: Streamlit, Gradio, React with Flask, FastAPI.
- Mobile development: Flutter with TensorFlow Lite, React Native with VisionCamera, Android CameraX with ML Kit.
- Version control: Git with GitHub or GitLab.

9 Academic Integrity

All code and models must be the original work of the group. Pre-trained models and publicly available libraries are permitted provided they are clearly cited. Plagiarism, including copying another group's code or report, will result in a mark of zero and a formal academic misconduct report in accordance with the UOWD Academic Integrity Policy.